Mobile Demo Script Explanation -- Keyboard (Exploitation, Challenge 04)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: A Zip Slip path-traversal bug in the update mechanism lets a malicious zip write into the target's own SharedPreferences file.

Background -- what the terms below mean

What does "Exploitation" mean, as a category? An **Exploitation** challenge gives you a real target Android app with an actual security mistake in it, and asks you to write a second, small app of your own that takes advantage of that mistake to get the flag.

What is a vulnerability? A mistake in how software was built that lets someone do something the developer never intended -- here, get the flag without going through the app the normal, intended way.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**) -- to find the mistake in the first place.

What is a package name? Every Android app has a unique ID string (like org.naija.fdexploit). Android refuses to install two different apps that both use the exact same ID, which occasionally trips up a first submission attempt if the exploit app accidentally reuses the target's own package name.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/04 Keyboard\Original Target APK.apk
```

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 04 Keyboard.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

Zip Slip: the target polls /sdcard/update.zip every 5 seconds and extracts it with zero path-traversal sanitization. Our crafted zip traverses into the target's own shared_prefs XML to set debugmode=true, then deliberately fails a second entry to trigger the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

Files to open

1. Open the target APK in jadx-gui:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/04 Keyboard\Original Target APK.apk
```

2. MainActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/04 Keyboard\Android Studio Project\app\src\
```

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found it polls /sdcard/update.zip every 5 seconds and extracts it with no path-traversal sanitization (a Zip Slip vulnerability).

2. I crafted a malicious zip whose entry names traverse out of the extraction folder and into the target's own SharedPreferences XML file, setting debugmode=true.

3. I made a second entry in the zip deliberately fail extraction, triggering the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

4. I built and submitted the APK; the platform's analysis log confirmed the debug dump revealed the flag.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

```
MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```

Source: Verbatim from /apk/81695 analysis log

The real, verbatim output this run produced:

```
dropped malicious update.zip at /storage/emulated/0/update.zip (670 bytes)
Exception while extracting the zip: java.io.FileNotFoundException: .../GlobalKeyboardPrefs.xml/x (Not a dire
dumping all info for debugging
debugmode: true
author: reyammer
flag: MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```